HACEC: Efficient and Auditable Anonymous Access Control for Edge Computing Services

Jie Chen, Haodi Zhang, Shuai Wang, and Huamin Jin

***Abstract*—In recent years, edge computing has undergone significant growth, but ensuring anonymity, efficiency, and auditability in service utilization remains challenging. This article proposes HACEC, an efficient and auditable anonymous access control scheme for edge computing. HACEC utilizes a dual-token mechanism to achieve anonymity and efficient service utilization. It allows users to access services using temporary identities, without exposing their real identities. We integrate the Trusted Execution Environment (TEE) and threshold cryptography into this mechanism to prevent several attacks. This mechanism not only protects users' identity privacy but also avoids reliance on an always-online cloud, thereby addressing the issue of a single point of failure. Additionally, HACEC proposes an auditable Multi-Authority Attribute-Based Encryption (MABE) scheme and a hierarchical audit mechanism to achieve efficiency and security in audit. The system separates the audit process into data audit and identity tracing. Data audit would not expose users' identities and is performed efficiently through the MABE scheme. Once malicious behaviors are detected, identity tracing can be performed in a threshold manner to retrieve the identities. As a result, HACEC achieves a trade-off between efficiency, security, and auditability. We provide a comprehensive security analysis and performance evaluation to demonstrate the security and efficiency of HACEC.**

***Index Terms*—Edge computing, access control, audit, anonymity, attribute-based encryption.**

I. Introduction

OVER the last decade, cloud computing has rapidly emerged as a new centralized computing paradigm that allows users to outsource their intensive computing tasks to cloud servers [1], [2]. However, the growing demands of computationally intensive tasks present new challenges for cloud computing. Transmitting large amounts of data imposes significant bandwidth burdens on cloud servers. Furthermore, the long-distance round-trip data transmission introduces non-negligible latency, particularly for latency-sensitive applications such as autonomous driving [3], [4] and healthcare [5], [6]. In addition, temporary network outages further exacerbate these issues.

To address these challenges, edge computing has been proposed as a promising architecture that offloads computation tasks to the network edge [7], [8], [9]. By requesting computing resources from nearby edge nodes, users can effectively reduce request latency and alleviate network loads on the cloud. While edge computing offers significant convenience, it also introduces new challenges, such as data leakage [10]. Malicious or unauthorized edge servers may deceive users and illegally access their data. This issue is exacerbated by the open and untrusted nature of edge networks, potentially hindering widespread adoption if not properly mitigated.

Access control has been recognized as a key mechanism to protect user privacy [11], [12], [13]. A straightforward solution is to deploy an always-online authentication server in the cloud. However, this approach suffers from high latency due to multiple rounds of direct server-client communication and introduces a single point of failure, making it unsuitable for the edge computing environment.

Token-based solutions, such as JWT [14] and OAuth [15], efficiently address the above issue. The cloud issues tokens to users, enabling them to access edge services using these tokens, thereby reducing the dependence on the cloud. Recently, the APECS framework [16] was proposed for edge computing, leveraging Multi-Authority Attribute-Based Encryption (MABE) and a token-based mechanism to achieve mutual authentication between users and edge servers without requiring an always-online cloud server. Nevertheless, the potential privacy leakage arising from disclosed identity information remains insufficiently addressed. Tokens submitted to edge servers may reveal users' identities, raising privacy concerns. Identity protection is necessary. For example, in autonomous driving applications, vehicles must provide their geographic location to edge nodes to obtain information about surrounding vehicles and determine optimal routes. Without identity protection, attackers could track vehicles' historical routes, leading to privacy concerns.

An anonymous and auditable access control scheme, AADEC [17], was proposed to address the above challenges. To achieve anonymity, AADEC leverages cryptographic accumulators [18] and signatures of knowledge (SoK) [19] to generate proofs for user validation. This design allows users to authenticate themselves via the accumulator without revealing their identities, which are protected through SoK. In addition, AADEC employs an audit server to facilitate auditing, which is authorized to recover both users' requested content and identities.

Motivation. From a privacy perspective, AADEC relies on the trustworthiness of the audit server. If the audit server is compromised, an attacker could easily retrieve user identities, making it a single point of failure and undermining privacy. A

Received 22 August 2024; revised 20 August 2025; accepted 9 October 2025. Date of publication 14 October 2025; date of current version 12 March 2026. *(Corresponding author: Jie Chen.)*

The authors are with China Telecom Research Institute, Guangzhou 510660, China (e-mail: chenj135@chinatelecom.cn; zhanghaodi@chinatelecom.cn; wangshuai8@chinatelecom.cn; jinhm6@chinatelecom.cn).

Digital Object Identifier 10.1109/TDSC.2025.3621286

common mitigation strategy is to deploy multiple audit servers with a threshold mechanism, ensuring that compromising a few servers does not reveal user identities [20], [21]. However, this approach introduces computational overhead, as auditing each request requires collaboration among multiple servers. Furthermore, the combination of accumulators and SoK for anonymous authentication introduces another challenge. When a new user registers, the accumulator state is updated, forcing all existing users to update their own state before accessing services. These extra interactions increase latency and render the system more susceptible to temporary network outages.

Contribution. In this paper, we propose an efficient and auditable anonymous access control scheme for edge computing, dubbed HACEC, to address the challenges discussed above. HACEC improves efficiency in two key aspects: auditing and anonymous access control. To achieve efficient auditing, HACEC introduces a hierarchical audit mechanism, which separates the audit process into data audit and identity tracing. Data audit involves recovering users' request messages to detect potential malicious behaviors, whereas identity tracing focuses on retrieving the identities of users engaging in malicious actions. Data audit is handled by a single data audit server, while identity tracing is performed collaboratively by a group of identity servers using a threshold approach. This design not only enhances efficiency but also preserves user privacy, ensuring that users' identities remain protected even if a subset of servers is compromised.

Furthermore, HACEC incorporates an auditable Multi-Authority Attribute-Based Encryption (MABE) scheme with multiple Attribute Issuing Authorities (AIAs) to support efficient data audit. Each AIA manages a distinct attribute domain and issues decryption keys to edge servers. Users encrypt their requests using the MABE scheme, and only edge servers with the corresponding decryption keys can decrypt the ciphertexts. To enable efficient auditing, a tracing key is introduced, allowing the audit server to decrypt ciphertexts even without possessing the decryption keys issued by the AIAs.

To achieve both efficient anonymity and service utilization, HACEC adopts a dual-token mechanism consisting of long-term tokens and temporary tokens. During registration, users are issued long-term tokens linked to their real identities. Before accessing services, users interact with identity servers using their long-term tokens to obtain temporary tokens. We introduce the Trusted Execution Environment (TEE) to ensure the privacy of the long-term tokens and real identities, preventing identity servers from correlating temporary tokens with their real identities. The validation of long-term tokens takes place within the TEE of identity servers, ensuring that the servers do not gain access to users' long-term tokens. Furthermore, to enable identity tracing, users' identity information is embedded within the temporary tokens, allowing identity servers to retrieve it in a threshold manner. Other entities cannot extract any identity-related information from the token, nor can they link two different temporary tokens to the same user.

By integrating these techniques, HACEC retains all features of each component. We provide a comprehensive security analysis to demonstrate that HACEC is resilient against various attacks. We also conduct a comprehensive performance analysis to show the efficiency of HACEC. Compared with existing schemes (e.g., AADEC), HACEC offers stronger security guarantees while maintaining lower computation costs.

The remainder of this paper is organized as follows. Section II presents the related work. Section III provides the basic preliminaries. Section IV describes the system model and threat model. Section V gives an overview of HACEC. Section VI presents the construction of HACEC. Section VII provides the security analysis. Section VIII evaluates the performance of HACEC. Section IX concludes this paper.

II. Related Work

Edge computing is a distributed computing paradigm that places computation and data storage closer to the data source—such as sensors, mobile devices, or other end devices. Its primary goal is to reduce latency, bandwidth usage, and response time, while ensuring data security and privacy. Many works have been proposed to address the challenges of edge computing, such as service latency [22], [23], energy consumption [24], [25], and security and privacy [26], [27], [28]. In this paper, we mainly focus on the security and privacy issues of edge computing.

Access control is a widely adopted approach to address security and privacy challenges in edge computing [11], [12], [29]. Access control research has been extensively studied, and attribute-based access control (ABAC) is one of the most promising solutions [30], [31], [32].

ABAC is a flexible and dynamic access control model that grants authorization based on the attributes of the user, resource, and environment, and has undergone significant development in recent years. It allows for fine-grained access control, enabling authorized users to access precisely defined resources. ABAC schemes are generally implemented using attribute-based encryption (ABE) [33], [34], [35], which encrypts data based on specified attributes and ensures that the ciphertext can only be decrypted by recipients whose attributes satisfy the access policy. ABE is a type of public-key encryption that uses attributes as part of the encryption process. Within ABAC, ABE can be used to encrypt the resource or the access policy, allowing only authorized users with matching attributes to decrypt and access the resource. However, traditional ABE is confronted with security limitation: it relies on a central authority to issue private keys. If the authority is compromised, adversaries can generate arbitrary users' private keys at will, making the scheme insecure. Consequently, the central authority becomes a single point of failure.

Apart from ABAC, capability-based access control scheme (CapBAC) [36], [37] is an alternative access scheme. In most token-based CapBAC implementations, users are granted capabilities by the cloud, which represent specific permissions enabling them to access certain resources or perform certain actions. Recently, OAuth [15] was proposed to distribute tokens for authentication and authorization. Nevertheless, the access token does not contain user-specific information, allowing an attacker to obtain the privileges once the access token is hijacked by the attacker. To address this issue, Heracles [38] was proposed

to modify the payload of access tokens. However, it remains vulnerable if an unfaithful entity shares its token.

Recently, Dougherty et al. [16] proposed APECS, an access control framework for edge computing. APECS leverages multi-authority attribute-based encryption (MABE) to securely offload computing tasks to edge nodes and employs token-based authentication for user authentication and authorization. However, it raises privacy concerns caused by identity leakage, as the tokens submitted to edge nodes exposes users' identity information. To address this challenge, Zhou et al. [17] proposed an anonymous and auditable access control scheme, dubbed AADEC. To achieve identity hiding, AADEC combines the signature of knowledge (SoK) and cryptographic accumulator to generate identity proofs. When accessing a service, users submit the proof to edge servers without leaking their token. Additionally, AADEC utilizes an auditable MABE scheme to enable auditability. This method enables a trusted audit server which holds a tracing key to recover users' request messages and identities. Nonetheless, AADEC is still confronted with several challenges. First, its privacy relies on the audit server. Once the server is compromised, user identity confidentiality can no longer be guaranteed. Second, the accumulator is not well suited to edge computing environment, as users are required to update their membership proof generated by the accumulator to the latest state before requesting services. This requirement increases service access latency and introduces the risk of service unavailability caused by temporary network outages.

In this paper, we propose HACEC to address the above challenges, with the following key contributions:

- We design a hierarchical audit mechanism to address the single point of failure in the audit server. The mechanism divides the audit process into data audit and identity tracing. In the data audit phase, centralized data audit servers verify the legality of user requests. Only if malicious behavior is detected will the system trigger the identity tracing phase, which is executed by a set of identity servers in a threshold manner. This design eliminates the single point of failure in AADEC and achieves a better balance between security and efficiency.
- We propose an auditable multi-authority attribute-based encryption (MABE) scheme to enable efficient data audits. The scheme introduces a trusted authority holding an audit key. When users encrypt their requests, the ciphertext can be decrypted not only by authorized edge servers with the corresponding decryption keys, but also by the audit server holding the audit key, allowing the requested content to be recovered when necessary.
- We propose a dual-token mechanism to replace the identity proof implemented by SoK and accumulator. This mechanism utilizes temporary tokens to conceal users' real identities and improves the computational performance.

III. Preliminary

A. Basic Theory

1) Bilinear pairing. Given two multiplicative groups G and G_T with order p, a bilinear pairing is a map: $e : G \times G \rightarrow G_T$, which satisfies three properties. Bilinearity: $\forall a, b \in Z_p, g_1, g_2 \in G : e(g_1{}^a, g_2^b) = e(g_1, g_2)^{ab}$. Non-degeneracy: $\exists g_1, g_2 \in G, e(g_1, g_2) \neq 1_{G_T}$. Computability: There exists an efficient algorithm to compute e.

2) Threshold cryptography. In the (t, n)-threshold cryptography, a secret is divided into n pieces, and each party has a piece. Any subset of at least t parties can collaboratively reconstruct this secret, while any coalition of fewer than t pieces can retrieve no information on the secret. The parameter t and n denote the threshold and the total number of pieces, respectively. Threshold cryptography is widely used, such as threshold signatures and threshold encryption.

B. Auditable Multi-Authority Attribute-Based Encryption

Multi-Authority Attribute-based Encryption (MABE) allows a sender to encrypt a message under η attribute sets (η denotes the number of authorities) and a specific access policy, such that the receiver can decrypt this message if they possess the attributes satisfying the access policy for each authority [39]. In this paper, we design a variant of the MABE scheme proposed by Lewko et al. [39] to support auditability. This variant enables an auditor holding a tracing key to decrypt users' requests. The proposed auditable MABE scheme consists of the following algorithm:

$pp \leftarrow$Setup(λ). With the security parameter λ, this algorithm generates the system parameter pp.

$(tsk, tpk) \leftarrow$TracingKeySetup(pp). Given the system parameters pp, this algorithm outputs a tracing key pair (tsk, tpk).

$(SK_k, PK_k) \leftarrow$ AuthSetup($\mathbb{A}_k^A$). Each authority AIA_k invokes this algorithm with initial attributes $\mathbb{A}_k^A$ to generate its private key sets and public key sets. SK_k denotes the generated private keys corresponding to the attributes. PK_k is the corresponding public key set. After all the authorities finish setting, the global public key is set as $M_{PK} = \{PK_k\}_{k\in[1,n]}$.

$S_k \leftarrow$ KeyGen($GID, SK_k, \mathbb{A}_k^E$). When an edge server registers with an authority AIA_k, it submits its attribute set $\mathbb{A}_k^E$. Then AIA_k invokes this algorithm to generate a decryption key set S_k corresponding to the attribute set $\mathbb{A}_k^E$ (It can be viewed that $\mathbb{A}_k^E \in \mathbb{A}_k^A$). It takes inputs the global id of the edge server GID, the secret key of this authority SK_k, and the attribute set $\mathbb{A}_k^E$ that the edge server submits.

$CT \leftarrow$Encrypt($M_{pk}, tpk, \mathbb{A}^c, M$). This algorithm is used to encrypt messages under the specified attribute set. It takes as input the global public key M_{pk}, the tracing public key tpk, an attribute set $\mathbb{A}^c$, and a message M, and outputs the ciphertext CT.

$M \leftarrow$ Decrypt($M_{pk}, CT, \{S_k\}_{k\in[1,\eta]}$). This algorithm is invoked to decrypt the message. It takes as input the global public key M_{pk}, the ciphertext CT, and the secret keys issued by all AIAs $\{S_k\}_{k\in[1,\eta]}$. It outputs M if the decryption is successful, otherwise, it returns errors. If the attributes associated with $\{S_k\}$ satisfy the access policy specified by CT, then the decryption can be successful.

$M \leftarrow$ Audit(CT, tsk). This algorithm takes a ciphertext CT and a data audit private key tsk as inputs. Its purpose is to decrypt the ciphertext and obtain the plaintext M. If the decryption

process is successful, the algorithm outputs M; otherwise, it returns errors.

C. Threshold ElGamal Encryption

The threshold ElGamal encryption scheme [40] is adopted in HACEC. A set of parties share a secret key in a threshold way, where each party holds a secret share. A sender utilizes the public key to encrypt a message and only a sufficient number (at least the threshold) of parties can collaboratively decrypt it. Note that during the encryption phase, a random number is selected to prevent some attacks. For convenience and security, we present a variant of the encryption algorithm in which the random number is explicitly provided as an input to the Encrypt algorithm.

$pp \leftarrow \text{Setup}(\lambda)$. Given a security parameter λ, this algorithm returns the system parameters pp.

$(SK_{eg}, PK_{eg}) \leftarrow \text{KeyGen}(pp)$. This algorithm takes the system parameters pp as input and outputs the encryption key pair (SK_{eg}, PK_{eg}). SK_{eg} is split into n fragments in a threshold way, such that each party holds a share $SK_{eg,i}$. The public keys PK_{eg} and $\{PK_{eg,i}\}_{1\le i\le n}$ are shared in the system.

$C \leftarrow \text{Encrypt}(M, PK_{eg}, r)$. This algorithm takes as input a message M, the public key PK_{eg}, and a random value r. It computes the ciphertext C using r as the random number under PK_{eg}.

$M \leftarrow \text{Decrypt}(C, SK_{eg,i})$. This algorithm takes a ciphertext C and a secret key share $SK_{eg,i}$ as input, and outputs a partially decrypted value c_i.

$M \leftarrow \text{Aggre}(\{c_i\}_{i\in[1,t]})$. This algorithm takes as input at least t partially decrypted values and outputs the plaintext M.

D. Threshold BLS Signature

In HACEC, the BLS threshold signature scheme [41] is adopted to generate users' temporary tokens in a threshold manner in the anonymous identity generation phase. Note that in other parts of the paper, signatures are also used for various purposes, such as signing requests. The signature algorithm can be any secure algorithm, including BLS.

The BLS threshold signature scheme encompasses the following algorithms.

$pp \leftarrow \text{Setup}(\lambda)$. Given a security parameter λ, this algorithm returns the system parameters pp.

$(sk, pk) \leftarrow \text{KeyGen}(pp)$. This algorithm takes as input the system parameters pp, and outputs the signing key pair (sk, pk). The secret key sk is split into several fragments in a threshold way, such that each party owns a fragment sk_i. The public keys $\{pk, \{pk_i\}_{1\le i\le n}\}$ are shared in the system.

$\sigma \leftarrow \text{Sign}(m, sk_i)$. This algorithm takes as input a signing key share sk_i and a message m, and outputs the signature σ.

$\{true, false\} \leftarrow \text{Verify}(m, \sigma, pk)$. This algorithm takes as input the message m, the corresponding signature σ and the public key pk. It returns true if σ is a correct signature of m under pk, otherwise returns false.

$\sigma \leftarrow \text{Aggre}(\{\sigma_i\}_{i\in[1,t]})$. This algorithm combines t sub-signatures into a valid signature. If successful, it returns σ, otherwise it returns errors.

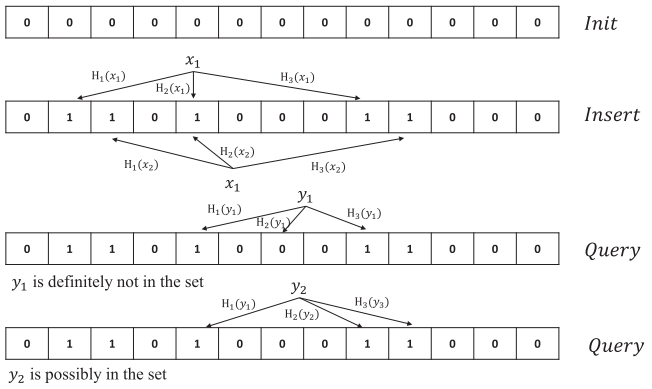

Fig. 1. An example of the Bloom filter using $\omega = 3$ and $\eta = 13$.

E. Bloom Filter

In our design, a user who registers with the service provider receives a long-term token that includes an expiration time. Upon receiving a request, a server can locally verify the validity of this token, without interacting with the service provider. However, certain users may need to be revoked before the token expires, due to reasons such as malicious behavior. Therefore, an efficient revocation algorithm is required to quickly determine whether a user's token has been revoked.

In this section, we introduce the Bloom filter (BF) to achieve this goal. A Bloom filter is a space-efficient probabilistic data structure that represents a set as a compact bit array. It supports membership queries with no false negatives (i.e., it can definitively determine that an element is not in the set), but may incur false positives (i.e., it may incorrectly indicate that an element is present). The key idea is to utilize ω hash functions $\{H_1, H_2, \ldots H_\omega\}$ to map an item x to a position corresponding to the compact array in the range $\{1, \ldots, \omega\}$. An item x is definitely not in the set if at least one of the corresponding bits in the array is 0. Fig. 1 illustrates an example of the Bloom filter. Note that the filter doesn't support deletion. If any item needs to be removed, the entire bit array must be reconstructed from the remaining items. The Bloom Filter consists of three algorithms.

$\{X, \{H_1, H_2, \ldots, H_\omega\}\} \leftarrow \text{BF.Setup}()$. Given the bit array length η and the number of hash functions ω, this algorithm outputs a bit array X of length η, initialized to 0, and a list of ω independent hash functions $\{H_1, H_2, \ldots, H_\omega\}$. Each hash function $H_i : \{0,1\}^* \to \{0, 1, \ldots, \eta - 1\}$ maps an arbitrary input to an index in the bit array.

$\{1, 0\} \leftarrow \text{BF.Insert(x)}$. This algorithm takes as input a value x and sets the bits at positions $H_1(x), H_2(x), \ldots, H_\omega(x)$ in X to 1. A bit position may be set to 1 multiple times, but only the first change has an effect. This algorithm outputs 1 when insertion is successful or 0 when errors occur.

$\{1, 0\} \leftarrow \text{BF.Query(y)}$.This algorithm checks whether y is contained in the set represented by X. Specifically, it computes the positions $H_1(y), H_2(y), \ldots, H_\omega(y)$ and checks the corresponding bits in X. If all bits are 1, the algorithm returns 1 (indicating y is possibly in the set); otherwise, it returns 0 (indicating y is definitely not in the set).

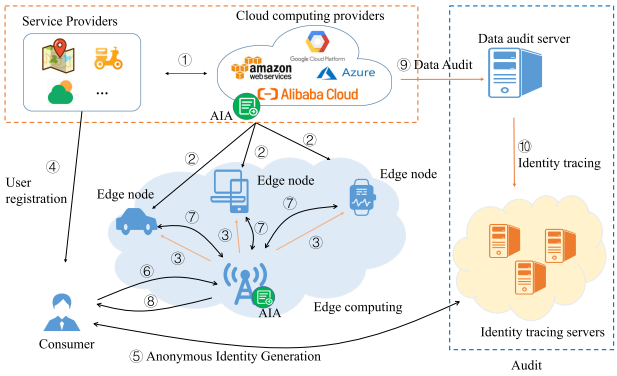

Fig. 2. System model.

F. Trusted Execution Environment

A Trusted Execution Environment (TEE) is a secure area of a processor that ensures the integrity and confidentiality of code and data loaded within it. It is isolated from the normal environment, known as the rich execution environment (REE), which includes operating systems such as Windows, Android, and iOS. By isolating execution, TEE provides strong protection against software-based attacks originating from the main OS. TEE enables secure execution of sensitive applications and handling of confidential data, safeguarding them from potentially malicious software, including the OS itself. Popular implementations of TEE include Arm TrustZone and Intel SGX, which are integrated into many modern CPUs.

In this paper, the TEE is employed to protect users' primary tokens during the anonymous identity generation phase. When users send their primary tokens to identity servers to obtain anonymous identities, key operations such as verifying the legitimacy of tokens and extracting relevant information are executed entirely within the servers' TEEs. Consequently, the operating system and any malicious processes cannot access or infer any information about the users' primary tokens.

IV. System Model and Threat Model

A. System Model

Our system model, as illustrated in Fig. 2, consists of three parts: the cloud environment, the edge environment, and the audit environment. The cloud computing environment includes service providers and cloud computing providers. The edge computing environment consists of base stations and edge nodes. The audit environment comprises data audit servers and identity tracing servers. Attribute Issuing Authorities (AIAs) are deployed on the cloud and base stations to issue decryption keys for edge nodes.

Service providers and cloud computing providers: A service provider offers a set of services, maintains registration information for both users and edge servers, and handles the users' revocation requests. But it doesn't have computing resources. A cloud computing provider provides central computing resources and allows the service provider to deploy its services on the cloud platform.

Resource consumers: A resource consumer is typically a user who is interested in the service provider's services. They can register with the service provider to obtain access rights to the offered services.

Base stations: A base station acts as a bridge between users and edge servers, with both connected to it. When a user accesses an edge service, the request is first sent to the base station, which forwards it to an edge server capable of providing the requested service.

Edge nodes: Edge nodes can be viewed as edge servers. They are located at the network edge and provide edge computing resources. They handle valid requests from the connected base station and send the result back. They need to register with the service provider to obtain authorization for handling users' requests.

Data audit servers (audit servers): A data audit server is responsible for auditing user requests. Since request messages are encrypted by users, the data audit server, which holds an audit key, can decrypt and retrieve the requests when necessary.

Identity tracing servers (identity servers): Identity tracing servers share an identity tracing key in a threshold manner, with each server holding a key share. Once a malicious behavior is detected by the data audit server, the identity tracing servers collaborate to recover the real identity associated with the behavior and submit it to the service provider.

Consider an intelligent transportation system in which vehicles periodically upload their location data to nearby base stations. The base stations forward the data to edge servers, which, based on the vehicle's current location, provide information about upcoming road conditions, such as traffic congestion. Compared to a traditional cloud-only approach, this design mitigates network latency and reduces bandwidth consumption. However, ensuring anonymity is critical: without it, edge servers could track users' identities and analyze their historical routes, resulting in serious privacy concerns.

System Entities Interactions. As shown in Step (1) of Fig. 2, a service provider initiates its services and AIA in the Cloud. Each base station likewise sets up its AIA. Each edge server registers with the service provider and base stations. Each AIA assigns specific attributes and decryption keys to the server based on the services it intends to provide (Step (2) and (3)). A user registers with the service provider, submits an interested service list, and receives a long-term access token (Step(4)). Before accessing a service, the user requests a temporary token from the identity servers for anonymity (Step(5)). With this token, the user encrypts his request message utilizing the required attributes of the service provider and his local base station and sends the ciphertext into the network via the base station (Step(6)). Upon receiving this request, the base station forwards it to one of its local edge servers. The edge server enforces the access control and sends the service execution result to the base station (Step(7)). The base station then relays the result to the user (Step(8)). In the audit phase, the data audit server is authorized to decrypt the request in order to verify the validity of user requests (Step(9)). If a malicious request is detected, this request will be forwarded to the identity servers, which collaboratively recover

TABLE I
NOTATIONS

$Notation$	Description
u	User
sp	The service provider
BS	The base station
ES	The edge server
ISs	The identity servers
AIA_k	The k-th attribute issuing authority
(usk, upk)	u's long-term key pair
(ask, apk)	u's temporary key pair
(tsk, tpk)	Data audit key pair
t_c	Current system time
$\sigma_{M,X}$	Signature of message M by entity X
GID	Global Identifier
ID_s	Service identifier
$[ID_s]$	List of service identifiers
$[\mathbb{A}^c]$	Attribute set used for encryption
$[A_e]$	List of MABE decryption keys pocessed by ES
(SK_{IS}, PK_{IS})	Identity tracing key pair
(sk_{IS}, pk_{IS})	Signing key pair of the BLS signature
$\mathcal{T}_L$	User u's long-term token issued by sp
$\mathcal{T}_T$	User u's temporary token issued by ISs
M_{pk}	Global public key in the MABE system

the real identity associated with the request and submit it to the service provider (Step (10)).

B. Threat Model

We consider two types of adversaries: external adversaries and internal adversaries.

1) External Adversaries: An external adversary aims to obtain the content requested by users through user-server interactions. The adversary can eavesdrop on messages transmitted over the open network and then attempts to decrypt or analyze them to retrieve sensitive information.
2) Internal Adversaries: An internal adversary can be a user, an edge server, a data audit server, or an identity server. A malicious user may attempt to access services despite having revoked privileges. A compromised edge server may attempt to break identity privacy by (i) extracting user identities from their requests or (ii) linking multiple requests from the same user. Additionally, an edge server with unauthorized or revoked privileges may impersonate a legitimate server to deceive users and steal requested content. A data audit server may also attempt to retrieve users' identities. A compromised identity server may collude with an edge server to retrieve users' identities.

V. Blocks and Overview

In this section, we give an overview of HACEC and its building blocks. Table I presents the primary notations mentioned in this paper.

A. User Registration and Authorization

A user u, who is interested in some of the service provider's sp services, must register with sp to obtain a long-term token. This token grants u access rights to these services offered by sp. It allows u to authenticate their privileges when accessing services. sp signs this token for integrity verification. A token is defined as $\mathcal{T} = (ID_p, [ID_s], upk, t_{exp})$, where ID_p denotes the identifier of sp, $[ID_s]$ is the list of service identifiers that u is authorized to access, upk is the public key of u's long-term identity, and t_{exp} specifies the expiration time of the token. Once this token expires, the user must reapply for a new token.

B. Anonymous Authentication

To achieve anonymity, HACEC employs two types of tokens: the long-term token and the temporary token. The long-term token, as discussed above, is issued by sp when u registers with sp. The temporary token, issued by identity servers, is anonymous and can be used when u requests services. Before accessing a service, u generates a fresh temporary identity and sends it together with the user's long-term token to the identity servers. Identity servers verify the long-term token and generate a temporary token for them in a threshold way. This temporary token has a structure similar to the long-term token. The temporary token is then used to authenticate service requests at edge servers, which process only requests accompanied by valid tokens.

However, submitting users' long-term tokens to identity servers during the temporary token generation phase raises concerns about identity leakage. Each identity server can potentially learn a user's real identity from their long-term tokens. Moreover, the linkage between the long-term token and the temporary token could allow malicious servers to collude with attackers to break the anonymity. As a result, the temporary token does not guarantee effective anonymity. To address this challenge, HACEC adopts Trusted Execution Environment (TEE) (as described in Section III) within the identity servers. The long-term token is validated inside the TEE, ensuring that identity servers learn nothing about the long-term tokens. This approach effectively protects users' identity privacy and prevents collusion between malicious identity servers and edge servers.

C. Audit

From the perspective of privacy and efficiency, HACEC divides the audit process into two parts: data audit and identity tracing. Data audit is executed periodically by a single audit server. Upon detecting a malicious request, the server forwards it to identity servers which collaborate in a threshold manner to recover the corresponding identity from this request. To facilitate these two types of audit, HACEC utilizes an auditable MABE scheme and ElGamal threshold encryption. MABE adopts a tracing key held by the data audit server, enabling it to recover the request message from the ciphertext. ElGamal threshold encryption employs a public/secret key pair, which is shared among the identity servers in a threshold manner, allowing them to decrypt users' identities collectively.

During anonymous identity generation, identity servers encrypt the user's real identity and embed it into the temporary token. When requesting a service, the user employs the MABE algorithm to encrypt his request and sends it along with the temporary token to the edge server. The edge server can verify

the privileges of this request via the temporary token, but cannot learn the user's real identity.

In the audit phase, the data audit server decrypts the request using the tracing key. If the request is malicious, the data audit server submits it to identity servers. Identity servers extract the user's identity ciphertext from the temporary token and decrypt it to retrieve the user's real identity in a threshold manner. The ElGamal encryption introduces randomness, preventing the audit server from linking multiple temporary tokens to the same user. This further enhances privacy protection and prevents correlation attacks.

D. Overview

We give an overview of HACEC, focusing on the challenges it addresses. HACEC tackles two primary challenges. The first challenge is the single point of failure caused by the centralized audit server. If this server is compromised, it could lead to the disclosure of users' identities. The second challenge is related to the efficiency of service access. Existing solution [17] introduces SoK and cryptographic accumulator to achieve anonymous service utilization. The accumulator requires users to frequently update their states, making them not entirely suitable for edge computing environments, especially for those latency-sensitive applications.

To address the first challenge, HACEC adopts a hierarchical audit mechanism that divides the audit process into two phases: data audit and identity tracing. Data audit is still performed by a centralized server, while identity tracing is conducted using a threshold approach. The audit phase begins with data audit, and only if malicious behavior is detected do identity servers collaborate in a threshold manner to recover the user's real identity. Further details can be found in Section V-C. Additionally, HACEC proposes an auditable MABE scheme to ensure efficient data audit. This scheme allows users to encrypt their requests, ensuring that only authorized edge servers satisfying the specified conditions can decrypt and recover the plaintext. The MABE scheme introduces a tracing key, enabling the data audit server to retrieve users' requested content from MABE ciphertext and detect malicious behavior.

To address the second challenge, HACEC introduces a dual-token mechanism consisting of long-term tokens and temporary tokens. The long-term tokens are issued by the cloud and represent users' privileges. When accessing edge services, users interact with identity servers using the long-term token to access temporary tokens. Edge servers can verify users' privileges from their temporary tokens, but cannot retrieve their real identities. This mechanism avoids heavy computational overload on both users and servers. Additionally, users do not need to request new temporary tokens before each service request, which is determined by the privacy policy, thereby reducing the latency of accessing services.

By integrating the above techniques, HACEC enables users to efficiently access edge services without revealing their identities. At the same time, it allows the audit server and identity servers to detect malicious user behaviors without relying on the security of a single server, thereby addressing the single point of failure.

E. Auditable Multi-Authority Attribute-Based Encryption

In this paper, we adopt the Multi-Authority Attribute-Based Encryption (MABE) scheme to encrypt user requests. This enables any eligible edge server to access user messages without requiring users to specify a particular server. The MABE scheme allows users to specify an access attribute policy when requesting services, and only the edge servers that satisfy this policy requirement can decrypt the messages. To enable auditability, we propose an auditable multi-authority attribute-based encryption scheme based on Lewko et al.'s scheme [39]. It introduces a tracing key, such that apart from edge servers, an audit server that holds this key can also decrypt users' requests to check the validity. The detailed algorithms are provided below.

$pp \leftarrow \mathrm{Setup}(\lambda)$. In the setup phase, with the security parameter λ, public parameter $pp = \{p, G, G_T, e, g, H\}$ is determined, where G and G_T are cyclic groups of prime order p, $e : G \times G \rightarrow G_T$ is the bilinear pairing, g is the generator of G with order p, $H : \{0,1\}^* \rightarrow Z_p$ is a secure hash function.

$\{tsk, tpk\} \leftarrow \mathrm{TracingKeySetup}(pp)$. In this phase, the system chooses a random tracing secret key $tsk \in Z_p$ and the public key is computed as $tpk = g^{tsk}$.

$(SK_k, PK_k) \leftarrow \mathrm{AuthSetup}(\mathbb{A}_k^A)$. For an authority AIA_k, it chooses two random exponents $\alpha_i, y_i \in Z_p$ for each attribute i belonging to the authority and publishes $PK_k = \{e(g,g)^{\alpha_i}, g^{y_i} \forall i\}$ as its public key. The secret key $SK_k = \{\alpha_i, y_i \forall i\}$ is kept locally.

$S_k \leftarrow \mathrm{KeyGen}(pp, GID, SK_k, \mathbb{A}_k^E)$. For an attribute set i in $\mathbb{A}_k^E$, the authority AIA_k computes:

$$K_{i,GID} = g^{\alpha_i} H(GID)^{y_i},$$

and the decryption keys issued by this authority are $S_k = \{K_{i,GID}\}_{i \in A_k^E \cap A_k^A}$.

$CT \leftarrow \mathrm{Encrypt}(M_{pk}, tpk, \mathbb{A}^c, M)$. To encrypt a message M, the sender first computes the LSSS access structure (A, ρ) according to the attributes $\mathbb{A}^c$. A represents an $n \times l$ matrix and ρ maps the row of A to an attribute of the attribute set. Then the sender chooses two random numbers $s, u \in Z_p$, a random vector $v \in Z_p^l$ with s as its first entry and a random vector $\omega \in Z_p^l$ with 0 as its first entry. Let λ_x denote $A_x \cdot v$ and ω_x denote $A_x \cdot \omega$ where A_x represents row x of A. For each row A_x of A, it chooses a random $r_x \in Z_p$. The ciphertext is computed as:

$$\begin{aligned} E_0 &= Me(g^u, tpk)^s, \\ E_{1,x} &= e(g^u, tpk)^{\lambda_x} e(g,g)^{\alpha_{\rho_x} r_x}, \\ E_{2,x} &= g^{r_x}, \\ E_{3,x} &= g^{y_{\rho_x} r_x} g^{\omega_x} \forall x, \\ E_4 &= e(g^s, g^u) \end{aligned}$$

$M \leftarrow \mathrm{Decrypt}(M_{pk}, CT, \{S_k\}_{k \in [1,\eta]})$. Assume that the ciphertext is encrypted under an access matrix (A, ρ). If the decryptor has the secret keys $\{K_{\rho(x),GID}\}$ for a subset of row A_x of A such that (1,0,...,0) is in the span of these rows, then the decryptor proceeds as follows. For each such x, the decryptor computes:

$$E_{1,x} \cdot e(H(GID), E_{3,x}) / e(K_{\rho(x),GID}, E_{2,x}) = e(g^u, tpk)^{\lambda_x} e(H(GID), tpk)^{\omega_x}$$

The decryptor then chooses constants $c_x \in Z_p$ such that $\sum_x c_x A_x = (1, 0, \ldots, 0)$ and computes :

$$\prod_x (e(g^u, tpk)^{\lambda_x} e(H(GID), g)^{\omega_x})^{c_x} = e(g^u, tpk)^s$$

(Recall that $\lambda_x = A_x \cdot v$ and $\omega_x = A_x \cdot \omega$, where $v \cdot (1, 0, \ldots, 0) = s$ and $\omega \cdot (1, 0, \ldots, 0) = 0$).

Then the message can be obtained as:

$$M = \frac{E_0}{e(g^u, tpk)^s}$$

$M \leftarrow \text{Audit}(CT, tsk)$. To recover the message M, the algorithm computes:

$$M = \frac{E_0}{E_4^{tsk}}$$

Discussion. There is no need to prove that $E_1, E_{2,x}, E_{3,x}$ and E_4 are well-formed using methods like zero-knowledge proofs. The correctness of $E_1, E_{2,x}$ and $E_{3,x}$ is a prerequisite for the receiver to decrypt the message. In other words, if the receiver can recover the message, these values are proven to be correct. Auditing is used to determine whether the encrypted message is malicious. If E_4 is not generated correctly, for example, if E_4 is a random number, the audit will fail. In this case, the message is considered malicious. In summary, any attempt to obstruct the audit process is deemed malicious.

Correctness proof. In the decryption phase,

$$\begin{aligned}
&E_{1,x} \cdot e(H(GID), E_{3,x}) / e(K_{\rho(x),GID}, E_{2,x}) \\
&= \frac{e(g^u, tpk)^{\lambda_x} e(g, g)^{\alpha_{\rho_x} r_x} e(H(GID), g^{y_{\rho_x} r_x} g^{\omega_x}))}{e(g^{\alpha_{\rho_x}} H(GID)^{y_{\rho_x}}, g^{r_x})} \\
&= \frac{e(g^u, tpk)^{\lambda_x} e(g, g)^{\alpha_{\rho_x} r_x} e(H(GID), g^{y_{\rho_x} r_x} g^{\omega_x}))}{e(g^{\alpha_{\rho_x}}, g^{r_x}) e(H(GID)^{y_{\rho_x}}, g^{r_x})} \\
&= \frac{e(g^u, tpk)^{\lambda_x} e(H(GID), g^{y_{\rho_x} r_x}) e(H(GID), g^{\omega_x})}{e(H(GID)^{y_{\rho_x}}, g^{r_x})} \\
&= e(g^u, tpk)^{\lambda_x} e(H(GID), g)^{\omega_x} \\
&\prod_x (e(g^u, tpk)^{\lambda_x} e(H(GID), g)^{\omega_x})^{c_x} \\
&= e(g^u, tpk)^{\sum_x \lambda_x c_x} e(H(GID), g)^{\sum_x \omega_x c_x} \\
&= e(g^u, tpk)^{\sum_x A_x \cdot v \cdot c_x} e(H(GID), g)^{\sum_x A_x \cdot \omega \cdot c_x} \\
&= e(g^u, tpk)^{(1,0,\ldots,0) \cdot v} e(H(GID), g)^{(1,0,\ldots,0) \cdot \omega} \\
&= e(g^u, tpk)^s e(H(GID), g)^0 \\
&= e(g^u, tpk)^s
\end{aligned}$$

Therefore, the decryptor can obtain the message as:

$$M = \frac{E_0}{e(g^u, tpk)^s}$$

In the audit phase, the audit server computes:

$$E_4^{tsk} = e(g^u, g^s)^{tsk} = e(g^u, g^{tsk})^s = e(g, tpk)^s,$$

Then it can recover M by computing:

$$M = \frac{E_0}{E_4^{tsk}}$$

VI. Construction

In HACEC, users, service providers, cloud computing providers, base stations, edge servers, data audit servers, and identity tracing servers are involved. In this section, for simplicity, we only consider a user u, a service provider sp, a cloud computing provider C, a base station BS, an edge server ES, a data audit server AS and a set of identity tracing servers ISs. We assume the following building blocks in HACEC: the MABE scheme (Setup, TracingKeySetup, AuthSetup, KeyGen, Encrypt, Decrypt, Audit), the ElGamal encryption scheme (Setup, KeyGen, Encrypt, Decrypt), the Bloom filter scheme (Setup, Insert, Query), the BLS signature scheme (Setup, KeyGen, Sign, Verify, Aggre).

A. System Setup and Registration

System setup. With the security parameter λ, the system generates the public parameters, including initiating all building blocks by invoking MABE.Setup, ElGamal.Setup, BF.Setup, BLS.Setup.

Service provider registration and AIAs Setup. Due to the convenience of the Cloud, the service provider sp deploys its services on the Cloud, such that any interaction with users or edge servers can be completed through the Cloud. Therefore, sp is required to register with the Cloud to bootstrap the services. Then, each AIA (sp and BS) initiates itself by running the MABE.AuthSetup algorithm, generating secret keys and publishing the corresponding public keys. Then the BLS signing key pair (sk_{IS}, pk_{IS}) is generated using BLS.KeyGen algorithm and shared among all identity servers, with each server holding a sub-secret key pair (sk_{IS_i}, pk_{IS_i}). Similarly, the ElGamal encryption key pair (SK_{IS}, PK_{IS}) is determined via ElGamal.KeyGen algorithm and each identity server holds a sub key pair (SK_{IS_i}, PK_{IS_i}). Subsequently, the data audit server invokes the MABE.TracingKeySetup algorithm to generate a tracing key pair (tsk, tpk). All the public keys are shared among the system entities.

Tables initialization. After that, the following tables are initiated.

- $userTable$. The service provider initiates a user table to record users' registration information.
- $serverTable$. The service provider and each base station initiate a server table to record edge servers' registration information, respectively.
- $revocationTable$. The service provider and each identity server initiate a revocation table to record revoked tokens of users, respectively.

User Registration. Before accessing services provided by edge servers, a user u must register with the service provider sp for authorization (See Algorithm 1). Suppose that u is interested in a set of services $[ID_s]$ offered by sp. u initiates his registration by generating a long-term key pair (usk, upk). Then he submits

Algorithm 1: User Registration.

Require: u's interested services $[ID_s]$
Ensure: u obtains the long-term token $\{\mathcal{T}_L, \sigma_{\mathcal{T}_L}\}$
At User u
1: generate a long-term key pair (usk, upk)
2: send $\{upk, [ID_s], user\_data\}$ to sp
At Service Provider sp
3: **if** sp accepts u's registration request **then**
4: $\mathcal{T}_L = (ID_p, [ID_s], upk, t_{exp})$
5: $\sigma_{\mathcal{T}_L} \leftarrow \text{Sign}(\mathcal{T}_L, sk_{sp})$
6: store $\{upk, user_{data}, \mathcal{T}_L\}$ in $userTable$
7: send $\{\mathcal{T}_L, \sigma_{\mathcal{T}_L}, M_{pk}\}$ to u
8: **else**
9: return error
10: **end if**

Algorithm 2: Edge Server Registration.

Require: the list of identifiers $[ID_s]$ of services ES would like to provide, the global ID of ES.
Ensure: ES obtains the decryption key list issued by each AIA.
At Edge Server ES
1: send $\{[ID_s], GID_{ES}\}$ to two AIAs (both sp and BS that ES connected)
At Service provider sp **& Base station**
2: $[A_e] \leftarrow \text{MABE.KeyGen}(GID_{ES}, tpk, SK_{BS}, [ID_s])$
3: store $\{GID_{ES}, [ID_s], [A_e]\}$ in $serverTable$
4: send $\{[A_e]\}$ to ES

$\{upk, [ID_s], user\_data\}$ to sp. $user\_data$ is used to create u's account, containing basic information for sp such as email. Upon receiving the registration request from u, sp retrieves the list of server identifiers $[ID_s]$. Then sp checks the validity of this registration. If the registration is valid, sp generates an access token $\mathcal{T}_L$ for u, which contains the identity of the service provider ID_p, the authorized service list $[ID_s]$, u's long-term identity upk, and the expiration time t_{exp}. sp signs this token using its signing key sk_{sp}. The provider sp further obtains the global public key M_{pk} of the MABE scheme. It stores u's information along with the issued token, and then sends a tuple, including $\mathcal{T}_L$, $\sigma_{\mathcal{T}_L}$, and M_{pk} to u.

Edge Server Registration. Before being added to the computing resource pool for service execution, an edge server ES must register with the system. This registration allows sp to identify which services ES can provide. Edge server registration starts with ES sending the list of identifiers, $[ID_s]$, of services that it intends to provide and its global ID, GID_{ES} to two AIAs (both sp and BS that ES is connected to), as described in Algorithm 2. Upon receiving the registration request from an edge server, each AIA verifies its validity. If the request is valid, the AIA executes the MABE key generation algorithm to generate a set of decryption keys corresponding to the service list $[ID_s]$. The AIA then stores the registration information in $serverTable$ and sends them to ES. With these decryption keys, ES can decrypt user request messages, so long as ES can provide the services that users attempt to access.

B. Service Request and Response

Before requesting services, a user u needs to request an anonymous temporary token. The validity period of this token can be configured according to the privacy policy-for example, it may be restricted to a single use, or remain valid for a fixed duration such as one hour or one day. Once the token expires, u is required to request a new temporary token.

1) Anonymous Identity Generation: The anonymous token is issued by identity servers in a threshold manner, as shown in Algorithm 3. Each identity server utilizes a trusted execution environment (TEE) to validate users' long-term tokens. First, u randomly generates a temporary key pair (ask, apk) and uses it to generate a message M_u, which includes the temporary public key apk, the expiration time of the temporary identity t_{tmp}, a random value r used for ElGamal encryption, and a service list $[ID_t]$ representing which services u intends to access with this token. Next, u constructs his request Req_{aig}, which consists of M_u, his long-term identity token $\mathcal{T}_L$, the signature of this token $\sigma_{\mathcal{T}_L}$, and a freshness time t_{aig}. It's important to note the distinction between t_{tmp} and t_{aig}: t_{tmp} indicates the expiration time of the temporary token, while t_{aig} specifies the freshness time of this request. u then signs the request using his long-term secret key usk and securely sends the request along with the signature to all identity servers.

The operations of an identity server IS_i can be divided into two steps: validation of the long-term token, which is performed within the TEE, and generation of the temporary token, which is executed in the REE.

- *TEE*: Upon receiving Req_{aig} and σ_{upk}, TEE extracts M_u, $\mathcal{T}_L$, $\sigma_{\mathcal{T}_L}$, t_{aig}, apk, t_{tmp}, r, $[ID_t]$, ID_p, $[ID_s]$, upk, and t_{exp}. It first verifies the freshness of the request by comparing its expiration time t_{aig} with current system time t_c. Then, it checks the validity of the temporary identity expiration time t_{tmp} to ensure it does not exceed the allowed maximum duration t_{thre}. Next, it confirms that the requested service list $[ID_t]$ is a subset of the authorized services $[ID_s]$, ensuring that u only requests registered and permitted services. It then verifies the request's signature using upk, and checks the request's validity and the registration status of user u. If all checks pass, the TEE determines whether upk has been revoked using the Bloom filter and $revocationTable$. If upk is not revoked, TEE encrypts upk using the ElGamal encryption algorithm and sends $\{ID_p, [ID_t], apk, C_u, t_{tmp}\}$ to REE.
- *REE*: Upon receiving the processed data from the TEE, the REE takes $ID_p, [ID_t], apk, C_u, t_{tmp}$ and computes a BLS sub-signature $\sigma_{\mathcal{T},i}$ on $\mathcal{T}_T$ using its secret key share sk_{IS_i}. The resulting sub-signature is then sent to u.

Upon receiving the sub-signature from IS_i, the user u computes the ciphertext C_u and the token $\mathcal{T}_T$ in the same way. Then u checks the validity of each sub-signature. Once t valid

Algorithm 3: Anonymous Identity Generation.

Require: u obtains the long-term token $\mathcal{T}_L$ issued by SP.
Ensure: u obtains the temporary token $\mathcal{T}_T$ and its corresponding signature $\sigma_{\mathcal{T}_T}$ issued by identity servers

At User u
1: generate a temporary identity (ask, apk).
2: set $M_u = (apk, t_{tmp}, r, [ID_t])$
3: set $Req_{aig} = \{M_u, \mathcal{T}_L, \sigma_{\mathcal{T}_L}, t_{aig}\}$
4: $\sigma_{upk} = \text{Sign}(\text{Req}_{aig}, usk)$
5: send $\{Req_{aig}, \sigma_{upk}\}$ to all identity servers

At Identity server IS_i

At TEE
6: receive $\{Req_{aig}, \sigma_{upk}\}$
7: extract $\{M_u, \mathcal{T}_L, \sigma_{\mathcal{T}_L}, t_{aig}\} \leftarrow Req_{aig}$, $\{apk, t_{tmp}, r, [ID_t]\} \leftarrow M_u$, $(ID_p, [ID_s], upk, t_{exp}) \leftarrow \mathcal{T}_L$
8: **if** $t_{aig} < t_c$ **then**
9: return error
10: **else if** $t_c + t_{thre} < t_{tmp}$ **then**
11: return error
12: **else if** $[ID_t] \subseteq [ID_s]$ **then**
13: return error
14: **else if** $false \leftarrow \text{Verify}(\text{Req}_{aig}, \sigma_{upk}, upk)$ **then**
15: return error
16: **else if** $false \leftarrow \text{Verify}(\mathcal{T}_L, \sigma_{\mathcal{T}_L}, pk_{sp})$, **then**
17: return error
18: **end if**
19: **if** $\text{BF.Query}(upk) = 1$ **then**
20: query upk from $revocationTable$
21: **if** upk is in $revocationTable$ **then**
22: return error
23: **end if**
24: **end if**
25: compute $C_u = \text{ElGamal.Encrypt}(upk, PK_{IS}, r)$
26: send $\{ID_p, [ID_t], apk, C_u, t_{tmp}\}$ to REE

At REE
27: receive $\{ID_p, [ID_t], apk, C_u, t_{tmp}\}$ from TEE
28: set $\mathcal{T}_T = \{ID_p, [ID_t], apk, C_u, t_{tmp}\}$
29: $\sigma_{\mathcal{T},i} = \text{BLS.Sign}(\mathcal{T}_T, sk_{IS_i})$
30: send $\sigma_{\mathcal{T},i}$ to u

At User u
31: Upon receiving $\sigma_{\mathcal{T},i}$, compute $C_u = \text{ElGamal.Encrypt}(upk, PK_{IS}, r)$
32: set $\mathcal{T}_T = \{ID_p, [ID_t], apk, C_u, t_{tmp}\}$
33: check the validity of $\sigma_{\mathcal{T},i}$ by invoking $\text{BLS.Verify}(\mathcal{T}_T, \sigma_{\mathcal{T},i}, pk_{IS_i})$
34: after receiving t valid subsignatures (denoted by $\sigma^*_{\mathcal{T},1}$, $\sigma^*_{\mathcal{T},2}$,..., $\sigma^*_{\mathcal{T},t}$), aggregate them into a valid signature $\sigma_{\mathcal{T}_T} = \text{BLS.Aggre}(\{\sigma^*_{\mathcal{T},i}\}_{1 \le i \le t})$
35: verify the correctness of $\sigma_{\mathcal{T}_T}$ by invoking $\text{BLS.Verify}(\mathcal{T}_T, \sigma_{IS,u}, pk_{IS})$
36: **if** The above check fails **then**
37: return error
38: **end if**
39: return $\{\mathcal{T}_T, \sigma_{\mathcal{T}_T}\}$

Algorithm 4: Service Request.

Require: u's requested service identifier ID_s
1: choose a symmetric encryption key K randomly,
2: $C_1 \leftarrow \text{Encrypt}_{\text{sym}}(M, K)$
3: $C_2 \leftarrow \text{MABE.Encrypt}(M_{pk}, \mathbb{A}^c, K)$
4: set $Req = \{\mathcal{T}_T, \sigma_{\mathcal{T}_T}, ID_s, C_1, C_2, t_{req}\}$
5: $\sigma_{req} \leftarrow \text{Sign}(Req, ask)$
6: send $\{Req, \sigma_{req}\}$

signatures are collected, u combines them using Lagrange interpolation to obtain a complete signature $\sigma_{\mathcal{T}_T}$ and verifies its correctness. If all verifications succeed, u obtains the temporary token $\mathcal{T}_T = \{ID_p, [ID_s], apk, C_u, t_{tmp}\}$ along with the signature $\sigma_{\mathcal{T}_T}$.

It is worth noting that the Bloom filter may cause false positives: when $\text{BF.Query}(upk) = 1$, it indicates that the queried user public key upk might be revoked, but this is not certain. Therefore, in Algorithm 3, we add an additional step to query the revocation table after $\text{BF.Query}(upk) = 1$. This approach effectively reduces frequent accesses to the revocation table, thereby improving query efficiency.

2) Service Request: Suppose that u requests an edge service denoted by ID_s using the anonymous token. First, u randomly chooses a symmetric encryption key K (e.g., AES), and encrypts the request message, producing ciphertext C_1. To securely transmit K to an authorized edge server, u encrypts K with the MABE scheme using the global public key M_{pk} and requisite attribute set $\mathbb{A}^c$ corresponding to ID_s, producing ciphertext C_2. The attribute set $\mathbb{A}^c$ denotes the specific service requested by u. Only edge servers providing the desired service can decrypt C_2 (See Section III-B).

Then, u encapsulates the anonymous token $\mathcal{T}_T$, its signature $\sigma_{\mathcal{T}_T}$, the requested service identifier ID_s, the ciphertext C_1, C_2, and the freshness time t_{req} into a request Req. To ensure freshness, integrity, and authentication, u signs Req with his anonymous secret key ask. Then he sends the signed request to the base station, which forwards it to the edge network. The detailed process is provided i n Algorithm 4.

3) Service Response: Algorithm 5 outlines the process of the service response. When an edge server ES receives a server request from the base station, it extracts the necessary information, including the request's signature σ_{req}, the temporary token $\mathcal{T}_T$, the token's signature $\sigma_{\mathcal{T}_T}$, the requested service ID_s, the MABE ciphertext C_1 and C_2, the freshness time of this request t_{req}, the service provider identifier ID_p, the registered services $[ID_t]$, the temporary public key apk, the long-term identity ciphertext C_u, and the expiration time of the temporary token t_{tmp}. ES then proceeds with the following validations and verifications.

First, ES verifies the validity and freshness of the request using u's temporary public key apk and the request time t_{req}. If the signature verification fails, ES returns an error and terminates the connection. Successful validation confirms that the request is

Algorithm 5. Service Response

receive $\{Req, \sigma_{req}\}$
extract $\{\mathcal{T}_T, \sigma_{\mathcal{T}_T}, ID_s, C_1, C_2, t_{req}\} \leftarrow Req$, $\{ID_p, [ID_t], apk, C_u, t_{tmp}\} \leftarrow \mathcal{T}_T$
if $t_{req} < t_c$ **then**
  return error
end if
if $false \leftarrow \text{Verify}(Req, \sigma_{req}, apk)$ **then**
  return error
end if
if $t_{tmp} < t_c$ **then**
  return error
end if
if $ID_s \notin [ID_t]$ **then**
  return error
end if
if $false \leftarrow \text{Verify}(\mathcal{T}_T, \sigma_{\mathcal{T}_T}, PK_{IS})$ **then**
  return error
end if
if $true \leftarrow \text{checkServiceAvailability}(ID_s)$ **then**
  $K \leftarrow \text{MABE.Decrypt}(C_2, [A_e])$
  $M \leftarrow \text{Decrypt}_{\text{sym}}(C_1, K)$
  return $\text{doService}(M)$
else
  $\text{forwardService}(Req, \sigma_{req})$
end if

correct, fresh, and generated by u. ES then checks the freshness of u's identity by comparing the identity expiration time t_{tmp} with the current system time t_c.

Next, ES ensures that u has permission to access the requested service by verifying that $ID_s \in [ID_t]$. It further validates u's identity by verifying the signature $\sigma_{\mathcal{T}_T}$ using the public key of the identity servers pk_{IS}. If any verification fails, ES returns an error and terminates the connection. In case of a failed validation, u can interact with the identity servers using his long-term token to request a valid and fresh temporary token.

Upon successful identity validation, ES checks if it can provide the requested service. If it is not capable, the request is forwarded to another edge server. Otherwise, it decrypts C_2 using its decryption keys $[A_e]$, obtained during registration, to retrieve the symmetric key K. Subsequently, ES utilizes K to decrypt C_1, obtaining the request message M. Finally, ES executes the requested service, encrypts the result, and sends it to the base station, which then forwards it to user u.

C. Audit

The audit process consists of two parts: data audit and identity tracing. Identity tracing is performed only if malicious behavior is detected during the data audit phase, and it is executed by identity servers in a threshold manner.

(a) *Data Audit*. The central data audit server extracts users' request messages which are encrypted using the MABE scheme. The server first recovers the symmetric key by executing $K = \text{MABE.Audit}(C_2, tsk)$, where tsk is a tracing secret key. Then, it utilizes K to decrypt the request message C_1 and verifies the requested content to detect malicious behavior. If any malicious behavior is detected, the identity ciphertext C_u embedded in the temporary token will be forwarded to the identity servers for identity tracing.

(b) *Identity Tracing*. Upon receiving C_u from the audit server, the identity servers jointly reconstruct the long-term identity from C_u by invoking ElGamal.Decrypt and ElGamal.Aggre and submit the recovered identity to the service provider sp.

D. Revocation

HACEC supports two types of revocation: user revocation and edge server revocation.

(a) *User Revocation*. User revocation can be conducted by the service provider sp in two cases.

- Malicious behaviors. If any malicious behavior is detected, the identity servers can recover the corresponding public key and submit it to sp.
- User unsubscription. If a user decides to unsubscribe from the service, they can send an unsubscription request to sp.

To revoke u from the system, the service provider sp queries u's corresponding long-term token $\mathcal{T}_L$ and removes it from $userTable$. sp then adds $\mathcal{T}_L$ to $revocationTable$. Subsequently, sp sends $\mathcal{T}_L$ to all identity servers. Each identity server adds $\mathcal{T}_L$ to its local $revocationTable$ and inserts $\mathcal{T}_L$ into the Bloom filter using BF.Insert. If $\mathcal{T}_L$ is expired, the server can remove it from $revocationTable$. Note that the Bloom filter doesn't support deletion operations. Therefore, when a large number of tokens have expired, the Bloom filter should be reconstructed to maintain its efficiency and accuracy.

(b) *Edge Server Revocation.* Edge server revocation ensures that revoked servers cannot access users' request data. The revocation is handled by the base stations, which executes the MABE algorithm to regenerate new key pairs. This ensures that the revoked server, which holds the old keys, cannot decrypt subsequent requests. The revocation process involves two steps:

1) The base station BS (connected to the edge server) updates its MABE public/secret key pair and shares the new public key to recomputate the MABE global key.
2) BS then executes MABE.KeyGen to regenerate decryption keys for all connected edge servers, except the revoked one.

VII. Security Analysis

In this section, we do not consider adversaries who might interfere with the registration phase (including both user and edge server registration). All communications during the registration phase are well protected, typically achieved through a TLS connection. We assume that all adversaries are PPT adversaries. Based on the threat model outlined in Section IV-B, we analyze the security of HACEC from the perspectives of external adversaries and internal adversaries.

A. Security Model

We formalize the security analysis via different experiments performed by the adversary $\mathcal{A}$ and the challenger $\mathcal{C}$. $\mathcal{A}$ can query the following oracles provided by $\mathcal{C}$.

$\mathcal{O}_{\text{ureg}}$: User registration oracle. Upon receiving an identity upk and an interested service list $[ID_s]$ from $\mathcal{A}$, the challenger runs the user registration algorithm and returns $\{\mathcal{T}_L, \sigma_{\mathcal{T}_L}, M_{pk}\}$ to $\mathcal{A}$.

$\mathcal{O}_{\text{encMABE}}$: MABE Encryption oracle. Upon receiving a message m from adversary $\mathcal{A}$, the challenger $\mathcal{C}$ runs the MABE encryption algorithm to generate a ciphertext $c = (E_0, E_1, E_2, E_3, E_4)$ corresponding to m under the public parameters. The challenger returns the ciphertext c to $\mathcal{A}$.

$\mathcal{O}_{\text{anonToken}}$: Anonymous token oracle. Upon receiving from adversary $\mathcal{A}$ a service list $[ID_s]$, a public key upk, an anonymous public key apk, and a freshness time t_{tmp}, the challenger $\mathcal{C}$ runs the anonymous token generation algorithm to produce an anonymous token $\{\mathcal{T}_T, \sigma_{\mathcal{T}_T}\}$ and returns it to $\mathcal{A}$.

$\mathcal{O}_{\text{privKey}}$: Private key oracle. Upon receiving from adversary $\mathcal{A}$ a list of indices $\chi = \{i_1, i_2, \ldots, i_{t'}\}$, where $t' < t$, the challenger $\mathcal{C}$ retrieves the corresponding pre-generated partial private keys $\{sk_{i_1}, sk_{i_2}, \ldots, sk_{i_{t'}}\}$ and $\{SK_{i_1}, SK_{i_2}, \ldots, SK_{i_{t'}}\}$ to $\mathcal{A}$, where sk_{i_j} denotes the ElGamal threshold encryption private key and SK_{i_j} denotes the BLS signing key associated with index i_j.

B. Resisting External Adversaries

In HACEC, although users interact with edge servers without secure channels during the service request phase, the content of these interactions is protected by a symmetric encryption algorithm (i.e., AES). During the initial interaction, the user selects and encrypts a symmetric encryption key using the MABE scheme. All subsequent communications between the two entities are then encrypted using this symmetric key. Therefore, the security of the users' requested content depends on the security of the MABE scheme. The goal of external adversaries is to retrieve users' requested content by eavesdropping on messages transmitted over the public network. We now analyze the security of the proposed MABE scheme.

Lemma 1: If Lewko's scheme is IND-CPA secure, the proposed auditable MABE scheme is IND-CPA secure.

Proof: If an adversary can break the security of the proposed MABE, we can construct a simulator $\mathcal{S}$ to break the security of Lewko's scheme [39].

We construct a simulator $\mathcal{S}$ that interacts with the challenger $\mathcal{C}$ of Lewko's scheme and the adversary $\mathcal{A}$ as follows:

1) Setup. $\mathcal{C}$ sets up the environment of Lewko's scheme. $\mathcal{S}$ receives $(pp, M_{PK}, \mathbb{A}_k^E)$ from $\mathcal{C}$. Then $\mathcal{S}$ invokes MABE.TracingKeySetup(pp) to generate the audit key pair (tsk, tpk). $\mathcal{A}$ is given $(pp, M_{PK}, \mathbb{A}_k^E, tpk)$.
2) Query. $\mathcal{A}$ adaptively issues $\mathcal{O}_{\text{encMABE}}$ at most polynomially many queries. In each query, $\mathcal{A}$ chooses a plaintext m. $\mathcal{S}$ responds this query as follows.
   - $\mathcal{S}$ forwards m to $\mathcal{C}$ and obtains c:

$$C_0 = Me(g,g)^s,$$
$$C_{1,x} = e(g,g)^{\lambda_x} e(g,g)^{\alpha_{\rho_x} r_x},$$
$$C_{2,x} = g^{r_x},$$
$$C_{3,x} = g^{y_{\rho_x} r_x} g^{\omega_x} \forall x$$

   - $\mathcal{S}$ constructs the full ciphertext $c' = (E_0, E_1, E_2, E_3, E_4)$ and sends it to $\mathcal{A}$:

$$u \xleftarrow{r} Z_p,$$
$$tsk = u^{-1}, tpk = g^{tsk},$$
$$E_0' = C_0, E_1' = C_1,$$
$$E_2' = C_2, E_3' = C_3,$$
$$E_4' = C_0^u$$

3) Challenge. $\mathcal{A}$ selects two message M_0 and M_1, and sends them to $\mathcal{S}$. $\mathcal{S}$ forwards them to $\mathcal{C}$.
4) $\mathcal{C}$ randomly chooses $b \in \{0,1\}$. $\mathcal{C}$ and $\mathcal{S}$ computes the ciphertext $c^* = (E_0^*, E_1^*, E_2^*, E_3^*, E_4^*)$ to $\mathcal{A}$ over M_b following step 2. Then $\mathcal{S}$ returns c^* to $\mathcal{A}$.
5) When $\mathcal{A}$ outputs a guess b', $\mathcal{S}$ outputs b' as its own guess for the Lewko challenge.

Clearly, $c = (C_0, C_{1,x}, C_{2,x}, C_{3,x})$ is a valid ciphertext generated by Lewko's scheme, as $\mathcal{C}$ constructs it according to this scheme. Now, we prove that the structure of c' is consistent with that of our scheme within the simulation environment. As $tsk = u^{-1}, e(g^u, tpk) = e(g,g)$. c' can be simplified as follows:

$$E_0' = Me(g^u, g^{tsk}),$$
$$E_1' = e(g^u, g^{tsk})^{\lambda_x} e(g,g)^{\alpha_{\rho_x} r_x},$$
$$E_{2,x}' = g^{r_x},$$
$$E_{3,x}' = g^{y_{\rho_x} r_x} g^{\omega_x} \forall x,$$
$$E_4' = M^u e(g^s, g)^u = M^u e(g^s, g^u)$$

Although the ciphertext component E_4' in the simulation differs from E_4 in our proposed scheme by an additional coefficient M^u, this difference is effectively hidden as M and M^u can be treated as fixed constants. Regardless of the coefficient, the adversary can still effectively use its capability to decrypt the ciphertext c'. Therefore, the simulator $\mathcal{S}$ has successfully simulated the interaction environment for adversary $\mathcal{A}$, enabling $\mathcal{A}$ to operate with full capability as in the real attack scenario. This implies that if adversary $\mathcal{A}$ can break our proposed scheme with non-negligible advantage, then the simulator $\mathcal{S}$ can break the underlying Lewko scheme with the same advantage. This completes the proof. □

C. Resisting Internal Adversaries

An internal adversary can be a user, an edge server, a data audit server, or an identity tracing server.

1) Resisting Malicious Users: A malicious user aims to access edge services after their privileges have been revoked. The user holds multiple expired tokens and can interact with the cloud and identity servers according to the protocol. Since access to edge services requires a valid token, the user's only possible

strategy is to forge a new valid token based on the expired ones. We assume that the user can collude with up to $t'(t' < t)$ identity servers. We now prove that the user cannot forge a valid anonymous token even if they have multiple expired tokens.

Lemma 2: In HACEC, if threshold BLS algorithm is EU-CMA secure, given a set of expired primary tokens (i.e., long-term tokens) and temporary tokens, a PPT adversary $\mathcal{A}$ cannot forge a valid temporary token, even if he can collude with up to $t'(t' < t)$ identity servers.

Proof: If there exists an adversary $\mathcal{A}$ capable of forging a valid temporary token, we can construct a simulator $\mathcal{S}$ that can break the existential unforgeability under chosen message attack (EU-CMA) security of BLS algorithm (which has been proven secure [41]). We construct a simulator $\mathcal{S}$ that interacts with the BLS challenger $\mathcal{C}$ and the adversary $\mathcal{A}$, as follows:

1) $\mathcal{C}$ sets up the BLS environment, and generates the signing key pair (SK, PK) and its n shares $\{SK_1, SK_2, \ldots, SK_n\}$ using (t, n)-threshold secret sharing protocol. It also initiates a list $\mathcal{L}$ to save the queried messages. After that, it sends the public parameters and PK to $\mathcal{S}$.
2) $\mathcal{S}$ initiates HACEC using the above BLS environment as its signing module and generates a server-side encryption key pair (sk, pk) along with its n shares $\{sk_1, sk_2, \ldots, sk_n\}$ utilizing (t, n)-threshold secret sharing protocol.
3) Query phase I. $\mathcal{A}$ is allowed to query the private key oracle $\mathcal{O}_{\text{privKey}}$ only once. Then $\mathcal{S}$ replies this oracle as follows:
   a) Upon receiving $\chi = \{i_1, i_2, \ldots, i_{t'}\}$, where $t' < t$, $\mathcal{S}$ forwards χ to $\mathcal{C}$ which returns the corresponding subset of signing key shares $\{SK_{i_1}, SK_{i_2}, \ldots, SK_{i'_t}\}$. $\mathcal{S}$
   b) $\mathcal{S}$ sends both $\{sk_{i_1}, sk_{i_2}, \ldots, sk_{i_{t'}}\}$ and $\{SK_{i_1}, SK_{i_2}, \ldots, SK_{i'_t}\}$ to $\mathcal{A}$.
4) Query phase II. $\mathcal{A}$ can adaptively query $\mathcal{O}_{\text{ureg}}$, $\mathcal{O}_{\text{anonToken}}$. $\mathcal{S}$ responds these oracles as follows.
   - $\mathcal{O}_{\text{ureg}}$: $\mathcal{S}$ runs the registration algorithm to generate primary token for $\mathcal{A}$. The expiration time t_{exp} of the token is set such that $t_{\text{exp}} < t_{\text{c}}$, i.e., the token is already expired when issued to $\mathcal{A}$.
   - $\mathcal{O}_{\text{anonToken}}$: a) Upon receiving a service list $[ID_s]$, a public key upk, an anonymous public key apk, and a freshness time t_{tmp}, $\mathcal{S}$ computes $C_u = \text{ElGamal.Encrypt}(upk, pk, r)$, where r represents a random number. b) Then $\mathcal{S}$ sets $\mathcal{T} = \{ID_p, [ID_s], apk, C_u, t_{tmp}\}$, where ID_p is generated during the initiation phase, and sends $\mathcal{T}$ to $\mathcal{C}$. $\mathcal{C}$ computes $\sigma = \text{BLS.Sign}(\mathcal{T}, sk)$ and sends it back. $\mathcal{C}$ maintains a list $\mathcal{L}_T$ to records all the signing message. c) $\mathcal{S}$ returns $\{\mathcal{T}, \sigma\}$ to $\mathcal{A}$.
5) Challenge. $\mathcal{A}$ outputs an anonymous token $\{\mathcal{T}', \sigma'\}$, where $\mathcal{T}' = \{ID'_p, [ID_s]', apk', C'_u, t'_{tmp}\}$ to $\mathcal{S}$. The simulator $\mathcal{S}$ forwards $(\mathcal{T}', \sigma')$ to the challenger $\mathcal{C}$ of the BLS signature scheme as its forgery attempt. If σ' is a valid BLS signature on a message $\mathcal{T}' \notin \mathcal{L}_T$, then $\mathcal{S}$ successfully breaks the EUF-CMA security of the BLS signature scheme.

Obviously, in the above simulation environment, $\mathcal{T}$ and σ that the adversary obtains are identical to those in the real environment. Therefore, the adversary can fully utilize its capabilities. Although the adversary can obtain t' $(t' < t)$ subsigning keys, it cannot retrieve the origin signing key without sufficient subkeys. The subkeys have the same distribution with the origin key, so the difficulty of breaking a subkey is the same as breaking the origin key. If $\mathcal{A}$ can forge a valid token, the simulator $\mathcal{S}$ can break the EU-CMA security of BLS algorithm with the same probability. □

As a result, the adversary, without possessing a valid primary token, is unable to forge a legitimate anonymous token, and therefore cannot gain access to the edge services.

2) Resisting Edge Servers: An unauthorized edge server may impersonate a valid server to provide edge services, attempting to retrieve users' data or retrieve users' identities.

For the first case (i.e., retrieving data), we consider an edge server that does not possess any valid decryption keys issued by the authorities. We claim that the ability of such an edge server with respect to extracting the user's request content is no greater than that of an external passive adversary who only eavesdrops on the public channel.

Indeed, during a user–edge interaction the edge server only observes the MABE ciphertexts that an external eavesdropper would observe; it has no additional secret material that would enable decryption. By Lemma 1, an external adversary that only obtains MABE ciphertexts cannot descrypt the request content. Therefore the edge server likewise cannot extract any non-negligible information from the ciphertexts. Formally, if the edge server could distinguish two messages from their ciphertext with non-negligible advantage, we could construct an external adversary that contradicts Lemma 1.

We now prove that the second case (i.e., recovering identities) is also infeasible.

Lemma 3: In HACEC, if the threshold ElGamal encryption scheme is IND-CPA secure, then any PPT adversary $\mathcal{A}$ cannot retrieve a user's real identity from the user's temporary identity, even if it colludes with up to $t' < t$ identity servers.

Proof: If there exists an adversary $\mathcal{A}$ that can retrieve a user's real identity from the temporary identity, then we can construct a simulator $\mathcal{S}$ that breaks the IND-CPA security of the threshold ElGamal scheme. Specifically, $\mathcal{S}$ interacts with the challenger $\mathcal{C}$ of the ElGamal scheme and the adversary $\mathcal{A}$ as follows:

1) Setup. $\mathcal{C}$ initiates the ElGamal environment, and generates an encryption key pair (SK, PK) and its n shares $SK_1, SK_2, \ldots, SK_n$.
2) $\mathcal{S}$ initiates the HACEC environment, where the initiation of the ElGamal uses the environment created by $\mathcal{C}$. It then generates a signing key pair (sk, pk) and its n shares $\{sk_1, sk_2, \ldots, sk_n\}$ utilizing (t, n)-threshold secret sharing protocol. $\mathcal{S}$ sends the public parameters, pk and PK to $\mathcal{A}$.
3) Query phase I. $\mathcal{A}$ is allowed to query the private key oracle $\mathcal{O}_{\text{privKey}}$ only once. Then $\mathcal{S}$ replies this oracle as follows:
   a) Upon receiving $\chi = \{i_1, i_2, \ldots, i_{t'}\}$ from $\mathcal{A}$, where $t' < t$, $\mathcal{S}$ forwards χ to $\mathcal{C}$ which returns

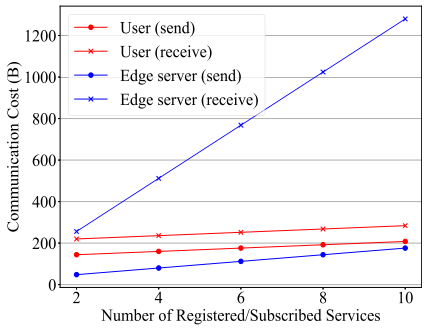

Fig. 3. Communication costs of users and edge servers in the registration phase.

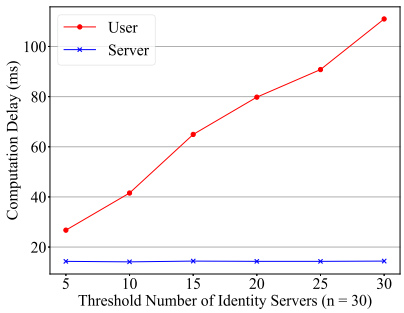

Fig. 4. Computation costs in the anonymous identity generation phase.

the corresponding subset of signing key shares $\{SK_{i_1}, SK_{i_2}, \ldots, SK_{i_t}\}$.

b) $\mathcal{S}$ sends both $\{sk_{i_1}, sk_{i_2}, \ldots, sk_{i_{t'}}\}$ and $\{SK_{i_1}, SK_{i_2}, \ldots, SK_{i_t}\}$ to $\mathcal{A}$.

4) Query phase II. $\mathcal{A}$ can adaptively query $\mathcal{O}_{\text{anonToken}}$ at most a polynomial number of times. $\mathcal{S}$ responds this oracle as follows.
   a) Upon receiving from adversary $\mathcal{A}$ a service list $[ID_s]$, a public key upk, an anonymous public key apk, and a freshness timestamp t_{tmp}, B forwards them to $\mathcal{C}$.
   b) $\mathcal{C}$ computes $C_u = \text{ElGamal.Encrypt}(upk, pk, r)$ and sends it back.
   c) Set $\mathcal{T} = \{ID_p, [ID_s], apk, C_u, t_{tmp}\}$, where ID_p denotes the ID of the cloud server, and t_{tmp} denotes the freshness time. $\mathcal{S}$ computes $\sigma = \text{BLS.Sign}(\mathcal{T}, sk_{IS})$.
   d) $\mathcal{S}$ returns $\{\mathcal{T}, \sigma\}$ to $\mathcal{A}$.
5) Challenge. $\mathcal{A}$ chooses three public keys pk'_0, pk'_1 and pk'_2, and a list of service $[ID'_s]$, where pk'_0 and pk'_1 represent two real identity public key and pk'_2 indicates the anonymous public key. $\mathcal{A}$ sends them to $\mathcal{S}$.
6) $\mathcal{S}$ sends pk'_0 and pk'_1 to the challenger $\mathcal{C}$. $\mathcal{C}$ randomly choose $b \in [0, 1]$, encrypts pk'_b, and returns the result e_b to $\mathcal{S}$.
7) $\mathcal{S}$ computes as follows:
   a) $C'_u = e_b$
   b) Set $\mathcal{T}' = \{ID_p, [ID'_s], pk'_2, C'_u, t'_{tmp}\}$, where t'_{tmp} denotes the freshness time.
   c) $\sigma' = BLS.Sign(\mathcal{T}', sk_{IS})$.
   d) Return $\{\mathcal{T}', \sigma'\}$ to $\mathcal{A}$.
8) $\mathcal{A}$ returns b' to $\mathcal{S}$. $\mathcal{S}$ outputs b' to $\mathcal{C}$ as the guess for b.

It is clear that adversary $\mathcal{A}$ can exercise its full capabilities since the simulation environment constructed by simulator $\mathcal{S}$ is indistinguishable from the real attack scenario faced by $\mathcal{A}$. Therefore, the above interaction constitutes a polynomial-time reduction from breaking the privacy of the temporary identity in HACEC to breaking the IND-CPA security of the threshold ElGamal encryption scheme. If the adversary $\mathcal{A}$ can distinguish the encrypted public keys with non-negligible advantage, then the simulator $\mathcal{S}$ can leverage this to break the IND-CPA security of the underlying encryption scheme with the same advantage. Since the threshold ElGamal encryption scheme is IND-CPA secure, it follows that $\mathcal{A}$'s advantage must be negligible. Hence, $\mathcal{A}$ cannot retrieve the user's real identity from the temporary identity. □

3) Resisting Data Audit Server and Identity Tracing Server: The goal of data audit servers and identity tracing servers is to retrieve users' real identities. Lemma 3 shows that a PPT adversary $\mathcal{A}$ cannot retrieve users' real identities from their temporary tokens, even if it can compromise up to $t-1$ identity tracing servers. Since data audit servers and identity tracing servers cannot get more information about users' real identities than concluding with $t'(t' < t)$ identity servers, HACEC is secure against these attacks.

VIII. Performance Evaluation

In this section, we treat TEE as a feasible and secure tool, and don't integrate it into our implementation. Therefore, all operations on the identity server side are executed in REE. We implement HACEC by using go language, DEDIS advanced Crypto library version 3.1.0, and GoFE library version 0.1.0. Our code is hosted on GitHub and can be found at [42].

The experiments are conducted on a laptop computer equipped with OS (Windows 10, 64-bit), CPU (i5-1135G7), and RAM (16 GB, DDR4), within China Telecom's large-scale innovation research infrastructures. The security level is set to 128 bits. The underlying pairing-friendly curve is the BN256 curve,[1] and the encryption curve is Curve25519. The signature algorithm is BLS signature. We don't use point compression to reduce bandwidth consumption. We evaluate the key components in the following subsections. All experimental results are averaged over 100 runs.

A. Evaluation of Registration

We evaluate the performance of user registration and edge server registration in this section. Since both users and servers don't have many computation tasks, we don't evaluate the computation costs. Instead, we focus on the communication costs, as illustrated in Fig. 3.

For users, the communication costs arise from sending/receiving registration information to/from the service provider. In our experimental setting, $user_{data}$ contains only an email address. For edge servers, the communication costs are

[1] It is important to note that the BN256 curve no longer operates at a 128-bit security level due to advancements in attacks. For stronger security, it is recommended to replace the BN256 curve with another curve, such as BLS12-381.

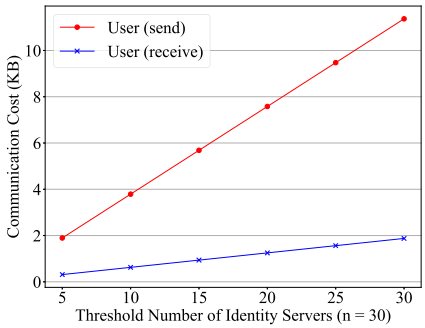

Fig. 5. Communication costs of users in the anonymous identity generation phase.

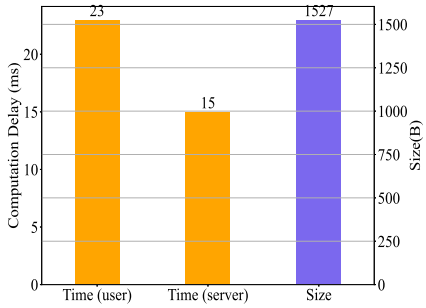

Fig. 6. Computation and communication costs in the service request and response phase.

incurred from interactions with both the service provider and the base station. It is worth noting that the size of data received by the edge server is greater than the amount of data sent by it, mainly due to the decryption keys issued by both AIAs. This leads to higher bandwidth consumption.

B. Evaluation of Anonymous Identity Generation

In the anonymous identity generation phase, a user interacts with identity servers to obtain sub-signatures and combines them in a threshold way to create a valid signature. In our experimental setup, the user is required to contact only a threshold number of identity servers rather than all of them. If an incorrect signature is detected, the user requests a sub-signature for another server. Therefore, increasing the threshold value will increase both the computation and communication costs of the user. Additionally, each service identifier is 8 bytes, and the user's token includes two service identifiers. The computation costs and communication costs are depicted in Figs. 4 and 5 respectively, with the parameter n set to 30.

The computation costs on the server side are primarily incurred from verifying the user's long-term token and signing the temporary token. The computation costs for the user are higher than those for the identity servers primarily due to verifying the correctness of sub-signatures and combining them into a valid signature. As shown in Fig. 4, the user takes around 120 ms to obtain a valid signature when the threshold value t is set to 30. In threshold cryptography, the values of t and n can be adjusted according to the desired security level and performance requirements. A larger t provides stronger security guarantees but it also leads to the increased computation and communication overhead, while a smaller t reduces overhead but lower the security level.

C. Evaluation of Service Request and Response

In the service request phase, the user generates an AES key to encrypt the request message and utilizes the MABE scheme to encrypt this key. The ciphertext, along with a signature, is then sent to the base station, which forwards them to the edge server. The computation costs are mainly incurred by the MABE encryption process and the communication costs primarily arise from the ciphertext generated by MABE.

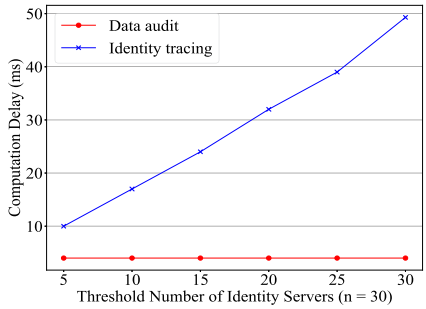

Fig. 7. Computation costs in the audit phase.

During the service response phase, upon receiving the request, the edge server decrypts it using its decryption keys obtained from the service provider and the base station. The computation costs on the server side consist of two parts: verifying the request and retrieving the request message from the ciphertext. The evaluation results are shown in Fig. 6. In our experiments, the size of the request message is less than 16 bytes (128 bits). Increasing the message length would proportionally raise communication costs, whereas the impact on computation costs would be negligible.

After that, the edge server executes the requested service, encrypts the results, and returns them to the user. The main overhead in this phase is caused by executing the service and returning the results; therefore, the performance of this part is not evaluated in this paper.

D. Evaluation of Audit

The audit process is divided into two parts: data audit and identity tracing. In the data audit phase, the central audit server recovers the symmetric key from the MABE ciphertext using the tracing key, and then decrypts the corresponding data. The computation costs on the data audit server are mainly caused by the MABE audit operation. In the identity tracing phase, the identity servers jointly decrypt the ciphertext contained in the user's temporary token to retrieve his identity. The computation costs on the identity servers are mainly incurred by the ElGamal threshold decryption process. The computation costs are shown in Fig. 7, where n is set to 30.

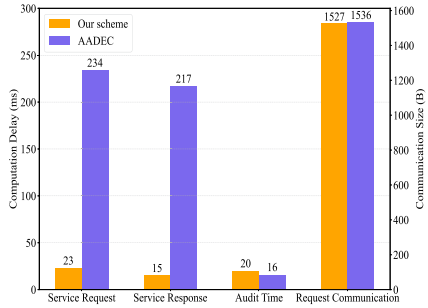

Fig. 8. Comparison between our scheme and AADEC.

E. Comparison

Fig. 8 presents the performance comparison between HACEC and the AADEC scheme. The x-axis corresponds to different types of overhead: service request computation, service response computation, audit computation, and service request communication, while the y-axis shows the measured values (milliseconds for computation, bytes for communication).

Compared with AADEC, HACEC exhibits lower computation overhead in the service request and response phases, while the communication overhead during the service request phase and the computation overhead in the audit phase are relatively similar. This can be partly attributed to differences in implementation details, hardware performance, and software optimizations; it also highlights the inherent efficiency of our proposed mechanisms. Although HACEC employs multiple components, the overall performance remains acceptable in practice.

IX. Conclusion

In this paper, we have proposed an efficient and auditable anonymous access control scheme for an edge computing environment, namely HACEC. HACEC is based on three key techniques: (1) a dual-token mechanism that ensures user identities remain anonymous during normal operations while allowing the conditional recovery of malicious users' identities when necessary; and (2) a hierarchical audit mechanism in which the data audit is performed by a single audit server to maximize efficiency, while identity tracing is carried out collaboratively by multiple identity servers in a threshold manner to enhance security; and (3) an auditable MABE scheme that enables users to encrypt their requests without establishing a peer-to-peer secure channel with edge servers, while allowing audit servers to efficiently recover these requests when required. We provide a comprehensive security analysis, demonstrating HACEC's resilience against various attacks, and perform experimental evaluations to assess its computation and communication efficiency.

While HACEC utilizes anonymous tokens to protect users' identities from being exposed, the process of obtaining these tokens introduces additional latency due to user interaction with identity servers. Although this impact can be minimized by (1) obtaining tokens in advance and (2) reducing the frequency of temporary token requests, it cannot be completely eliminated. Therefore, for future work, we will explore new approaches to achieve anonymity without significantly increasing access latency. Furthermore, to address the single point of failure issue, we deploy a set of identity servers that share an identity tracing key using threshold techniques. However, this approach introduces challenges in key management. In our future research, we will aim to find alternative solutions to address this issue.

References

[1] Y. Zhang, C. Xu, N. Cheng, and X. Shen, "Secure password-protected encryption key for deduplicated cloud storage systems," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 4, pp. 2789–2806, Jul./Aug. 2022.
[2] K. Yang, J. Shu, and R. Xie, "Efficient and provably secure data selective sharing and acquisition in cloud-based systems," *IEEE Trans. Inf. Forensics Secur.*, vol. 18, pp. 71–84, Oct. 2022.
[3] L. Liu, M. Zhao, M. Yu, M. A. Jan, D. Lan, and A. Taherkordi, "Mobility-aware multi-hop task offloading for autonomous driving in vehicular edge computing and networks," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 2, pp. 2169–2182, Feb. 2023.
[4] M. Wu, F. R. Yu, and P. X. Liu, "Intelligence networking for autonomous driving in beyond 5G networks with multi-access edge computing," *IEEE Trans. Veh. Technol*, vol. 71, no. 6, pp. 5853–5866, Jun. 2022.
[5] A. Kishor, C. Chakraborty, and W. Jeberson, "A novel fog computing approach for minimization of latency in healthcare using machine learning," *Int. J. Interactive Multimedia Artif. Intell.*, vol. 6, pp. 7–17, 2021.
[6] M. Hartmann, U. S. Hashmi, and A. Imran, "Edge computing in smart health care systems: Review, challenges, and research directions," *Trans. Emerg. Telecommun. Technol.*, vol. 33, no. 3, 2022, Art. no. e3710.
[7] J. Chen and X. Ran, "Deep learning with edge computing: A review," in *Proc. IEEE*, vol. 107, no. 8, pp. 1655–1674, Aug. 2019.
[8] K. Cao, Y. Liu, G. Meng, and Q. Sun, "An overview on edge computing research," *IEEE Access*, vol. 8, pp. 85714–85728, 2020.
[9] Y. Chen, F. Zhao, Y. Lu, and X. Chen, "Dynamic task offloading for mobile edge computing with hybrid energy supply," *Tsinghua Sci. Technol.*, vol. 28, no. 3, pp. 421–432, 2023.
[10] T. Li, X. He, S. Jiang, and J. Liu, "A survey of privacy-preserving offloading methods in mobile-edge computing," *J. Netw. Comput. Appl.*, vol. 203, 2022, Art. no. 103395.
[11] P. Ranaweera, A. D. Jurcut, and M. Liyanage, "Survey on multi-access edge computing security and privacy," *IEEE Commun. Surveys Tuts.*, vol. 23, no. 2, pp. 1078–1124, Second Quarter, 2021.
[12] F. Liu, G. Tang, Y. Li, Z. Cai, X. Zhang, and T. Zhou, "A survey on edge computing systems and tools," in *Proc. IEEE*, vol. 107, no. 8, pp. 1537–1562, Aug. 2019.
[13] C. Jiang, C. Xu, Y. Han, Z. Zhang, and K. Chen, "Two-factor authenticated key exchange from biometrics with low entropy rates," *IEEE Trans. Inf. Forensics Secur.*, vol. 19, pp. 3844–3856, Mar. 2024.
[14] M. Jones, J. Bradley, and N. Sakimura, "JSON web token (JWT)," IETF RFC 7519, May 2015. [Online]. Available: https://www.rfc-editor.org/rfc/rfc7519
[15] D. Hardt, "The OAuth 2.0 authorization framework," IETF RFC 6749, Oct. 2012. [Online]. Available: https://www.rfc-editor.org/rfc/rfc6749
[16] S. Dougherty, R. Tourani, G. Panwar, R. Vishwanathan, S. Misra, and S. Srikanteswara, "APECS: A distributed access control framework for pervasive edge computing services," in *Proc. 2021 ACM SIGSAC Conf. Comput. Commun. Secur.*, 2021, pp. 1405–1420.
[17] X. Zhou, D. He, J. Ning, M. Luo, and X. Huang, "AADEC: Anonymous and auditable distributed access control for edge computing services," *IEEE Trans. Inf. Forensics Secur.*, vol. 18, pp. 290–303, Nov. 2022.
[18] L. Reyzin and S. Yakoubov, "Efficient asynchronous accumulators for distributed PKI," in *Proc. Int. Conf. Secur. Cryptogr. Netw.*, 2016, pp. 292–309.
[19] M. Chase and A. Lysyanskaya, "On signatures of knowledge," in *Proc. Annu. Int. Cryptol. Conf.*, 2006, pp. 78–96.
[20] C. Jiang, C. Xu, Z. Zhang, and K. Chen, "SR-PEKS: Subversion-resistant public key encryption with keyword search," *IEEE Trans. Cloud Comput.*, vol. 11, no. 3, pp. 3168–3183, Third Quarter, 2023.
[21] Z. Zhang, C. Xu, C. Jiang, and K. Chen, "TSAPP: Threshold single-sign-on authentication preserving privacy," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 4, pp. 1515–1527, Jul./Aug. 2024.
[22] T. Bai, C. Pan, Y. Deng, M. Elkashlan, A. Nallanathan, and L. Hanzo, "Latency minimization for intelligent reflecting surface aided mobile edge computing," *IEEE J. Sel. Areas Commun.*, vol. 38, no. 11, pp. 2666–2682, Nov. 2020.

[23] M. Feng, M. Krunz, and W. Zhang, "Joint task partitioning and user association for latency minimization in mobile edge computing networks," *IEEE Trans. Veh. Technol*, vol. 70, no. 8, pp. 8108–8121, Aug. 2021.
[24] X. Chen, Y. Zhou, B. He, and L. Lv, "Energy-efficiency fog computing resource allocation in cyber physical Internet of Things systems," *IET Commun.*, vol. 13, no. 13, pp. 2003–2011, 2019.
[25] D. Zeng, L. Gu, and H. Yao, "Towards energy efficient service composition in green energy powered cyber–physical fog systems," *Future Gener. Comput. Syst.*, vol. 105, pp. 757–765, 2020.
[26] Y. Guo, H. Xie, C. Wang, and X. Jia, "Enabling privacy-preserving geographic range query in fog-enhanced IoT services," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 5, pp. 3401–3416, Sep./Oct. 2022.
[27] H. Nasiraee and M. Ashouri-Talouki, "Privacy-preserving distributed data access control for CloudIoT," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 4, pp. 2476–2487, Jul./Aug. 2022.
[28] Z. Tian et al., "Real-time lateral movement detection based on evidence reasoning network for edge computing environment," *IEEE Trans. Ind. Informat.*, vol. 15, no. 7, pp. 4285–4294, Jul. 2019.
[29] T. Wang, P. Wang, S. Cai, Y. Ma, A. Liu, and M. Xie, "A unified trustworthy environment establishment based on edge computing in industrial IoT," *IEEE Trans. Ind. Informat.*, vol. 16, no. 9, pp. 6083–6091, Sep. 2020.
[30] K. Xue et al., "A secure, efficient, and accountable edge-based access control framework for information centric networks," *IEEE/ACM Trans. Netw.*, vol. 27, no. 3, pp. 1220–1233, Jun. 2019.
[31] Y. Zhu, R. Yu, D. Ma, and W. Cheng-Chung Chu, "Cryptographic attribute-based access control (ABAC) for secure decision making of dynamic policy with multiauthority attribute tokens," *IEEE Trans. Rel.*, vol. 68, no. 4, pp. 1330–1346, Dec. 2019.
[32] M. Gupta, F. M. Awaysheh, J. Benson, M. Alazab, F. Patwa, and R. Sandhu, "An attribute-based access control for cloud enabled industrial smart vehicles," *IEEE Trans. Ind. Informat.*, vol. 17, no. 6, pp. 4288–4297, Jun. 2021.
[33] A. Sahai and B. Waters, "Fuzzy identity-based encryption," in *Proc. 24th Annu. Int. Conf. Theory Appl. Cryptogr. Techn.*, 2005, pp. 457–473.
[34] Y. Zhang, R. H. Deng, S. Xu, J. Sun, Q. Li, and D. Zheng, "Attribute-based encryption for cloud computing access control: A survey," *ACM Comput. Surv.*, vol. 53, no. 4, Aug. 2020, Art. no. 83.
[35] S. Banerjee, B. Bera, A. K. Das, S. Chattopadhyay, M. K. Khan, and J. J. Rodrigues, "Private blockchain-envisioned multi-authority CP-ABE-based user access control scheme in IIoT," *Comput. Commun.*, vol. 169, pp. 99–113, 2021.
[36] Z. N. Mohammad, F. Farha, A. O. M. Abuassba, S. Yang, and F. Zhou, "Access control and authorization in smart homes: A survey," *Tsinghua Sci. Technol.*, vol. 26, no. 6, pp. 906–917, 2021.
[37] Y. Nakamura, Y. Zhang, M. Sasabe, and S. Kasahara, "Capability-based access control for the Internet of Things: An ethereum blockchain-based scheme," in *Proc. 2019 IEEE Glob. Commun. Conf.*, 2019, pp. 1–6.
[38] Q. Zhou, M. Elbadry, F. Ye, and Y. Yang, "Heracles: Scalable, fine-grained access control for Internet-of-Things in enterprise environments," in *Proc. 2018 IEEE Conf. Comput. Commun.*, 2018, pp. 1772–1780.
[39] A. Lewko and B. Waters, "Decentralizing attribute-based encryption," in *Proc. 30th Annu. Int. Conf. Theory Appl. Cryptographic Techn.*, 2011, pp. 568–588.
[40] Y. Tsiounis and M. Yung, "On the security of ElGamal based encryption," in *Proc. Int. Workshop Public Key Cryptogr.*, 1998, pp. 117–134.
[41] D. Boneh, B. Lynn, and H. Shacham, "Short signatures from the weil pairing," in *Proc. Int. Conf. Theory Appl. Cryptol. Inf. Secur.*, 2001, pp. 514–532.
[42] HACEC Implementation, [GitHub repository], 2023. [Online]. Available: https://github.com/cj77821/HACEC-Implementation

Jie Chen received the master's degree from the University of Electronic Science and Technology of China, in 2022. He has published several papers in network security international journals, including the *IEEE Transactions on Information Forensics and Security*, *IEEE Transactions on Dependable and Secure Computing*. He is leading and participating in the development of several international standards. He is currently a researcher with China Telecom Research Institute. His research interests include data security, AI security, and blockchain.

Haodi Zhang received the master's degree from Sun Yat-sen University. She has led the development of multiple ITU-T international standards on security, published research in leading network security journals such as the *IEEE Transactions on Dependable and Secure Computing*, and invented several PCT-patented technologies in the field of cybersecurity. She is currently a researcher with China Telecom Research Institute. Her research interests include network security and AI-driven security.

Shuai Wang received the master's degree from Wuhan University. She is a senior engineer in cyberspace security and the director with the Cloud-Network Security Laboratory, China Telecom Research Institute. She is also a senior expert in network and information security with China Telecom. With 20 years of experience in the field of network security, her main research directions include cyberspace security simulation, network attack penetration, and network endogenous security.

Huamin Jin received the master's degree from Tsinghua University. He is a senior engineer in cyberspace security and a state special allowance expert. Now, he's deputy director (in charge) with the Cloud Network Security Lab, Research Institute, responsible for R&D, management and operation of the national cloud network infrastructure center. He was vice chair of TC8 (CCSA), member of Mobile Internet Sec. Nat'l Eng. Lab, adjunct prof. with Pengcheng Lab, and Dir. of Cybersecurity Inst., Research Institute. His research interest include cybersecurity.